**Reverse- and Reengineering**
**Homework 5**

Winter 2026/27 — Prof. Dr. Christian Hammer
Date: 17.11.2026 — Dr. Mohammad Rezaalipour

This sheet is not for submission. However, we strongly encourage you to solve the problems and bring your solutions to the tutorials for discussion.

## 1 Program Stack

Consider the x86-64 assembly program below, which implements a value-swapping mechanism:

1. Draw the call stack at the point of the call to `swap`.
2. Show the relationship between the formal parameters of `swap` and the actual parameters passed from `main`.

Listing 1: `swap` function

```
swap(int*, int*):
    push   rbp
    mov    rbp, rsp
    mov    QWORD PTR [rbp-24], rdi
    mov    QWORD PTR [rbp-32], rsi
    mov    rax, QWORD PTR [rbp-24]
    mov    eax, DWORD PTR [rax]
    mov    DWORD PTR [rbp-4], eax
    mov    rax, QWORD PTR [rbp-32]
    mov    edx, DWORD PTR [rax]
    mov    rax, QWORD PTR [rbp-24]
    mov    DWORD PTR [rax], edx
    mov    rax, QWORD PTR [rbp-32]
    mov    edx, DWORD PTR [rbp-4]
    mov    DWORD PTR [rax], edx
    nop
    pop    rbp
    ret
```

Listing 2: `main` function

```
main:
    push   rbp
    mov    rbp, rsp
    sub    rsp, 16
    mov    DWORD PTR [rbp-4], 5
    mov    DWORD PTR [rbp-8], 7
    lea    rdx, [rbp-8]
    lea    rax, [rbp-4]
    mov    rsi, rdx
    mov    rdi, rax
    call   swap(int*, int*)
    mov    eax, 0
    leave
    ret
```

## 2 Intermediate Representation (IR) Assembly

Consider the following program:

```
1  int foo(int n) {
2      int a = 0, b = 1, c = 0, i;
3      if (n > 1)
4          goto L2;
5      c = a;
6      goto L6;
7  L2:
8      if (n != 2)
9          goto L3;
10     c = b;
11     goto L6;
12 L3:
13     i = 3;
14     goto L4;
15 L4:
16     if (i <= n)
17         goto L5;
18     goto L6;
19 L5:
20     c = a + b;
21     a = b;
22     b = c;
23     i = i + 1;
24     goto L4;
25 L6:
26     return c;
27 }
```

1. Draw the unstructured control flow graph (CFG), treating each statement as a node.
2. What patterns have you identified in the CFG?
3. Show each step of the iterative refinement of the CFG.
4. Write the equivalent structured program without using `goto` statements.